SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

Symbiosis Institute of Technology, Pune

Department of Artificial Intelligence and Machine Learning

EXPERIENTIAL LEARNING · LABORATORY / COURSE REPORT

Hadoop MapReduce Paradigm

Big Data Analytics — hands-on MapReduce processing

STUDENT DETAILS (for official submission)

NAME	**Sukhada Dhingra**
PRN	**24070126505**
PROGRAMME	**B.Tech. AIML**
COURSE / PAPER	**Big Data Analytics**
ACADEMIC YEAR	**2025–26**
DATE OF SUBMISSION	**09 September 2026**
EMAIL	**dhingrasukhada@gmail.com**

Apache Hadoop 3.5.0 · OpenJDK 21 · Local (standalone) MapReduce

Jobs executed: WordCount | SalesAnalysis (units by category)

Confidential academic submission · original work of the student named above

1. Certificate / Declaration

I, Sukhada Dhingra, student of B.Tech. (Artificial Intelligence and Machine Learning) at Symbiosis Institute of Technology, Pune, constituent of Symbiosis International (Deemed University), hereby declare that the work presented in this experiential learning report titled “Experiential Learning: Hadoop MapReduce Paradigm” is my original work carried out as part of the course Big Data Analytics during the academic year 2025–26. The MapReduce programs were compiled and executed on Apache Hadoop 3.5.0 (standalone / local job runner). Screenshots attached in this PDF are captured from that execution and have not been copied from another student.

I further declare that I have cited standard references (Apache Hadoop documentation, Dean & Ghemawat MapReduce paper, White’s Hadoop: The Definitive Guide) wherever theory is summarised, and that no part of this report has been submitted elsewhere for academic credit.

Student Sukhada Dhingra PRN: 24070126505 Date: 09 September 2026 Signature: _____	Course Faculty Name: _____ Designation: _____ Date: _____ Signature: _____

2. Abstract

This experiential learning report demonstrates the Hadoop MapReduce programming paradigm for Big Data Analytics. Two jobs were implemented in Java, compiled against Hadoop 3.5.0, and executed with the local job runner. The first job is the canonical WordCount application: mappers emit (word, 1) pairs, a combiner performs local aggregation, and the reducer produces global frequencies. The second job, SalesAnalysis, aggregates units sold by product category from a CSV dataset — a typical analytics pattern (GROUP BY + SUM). The report explains HDFS, YARN and the map–shuffle–reduce pipeline, presents source code, and attaches screenshots of compilation, job counters, and `part-r-00000` output. On the WordCount corpus the job processed 34 input records, emitted 316 map records, reduced them (via combiner) to 197 keys, and wrote 197 output records. SalesAnalysis reduced 20 mapped rows to three category totals: grocery 170, electronics 125, clothing 82.

3. Contents

§	Section
1	Certificate / Declaration
2	Abstract
3	Contents
4	Aim, objectives and apparatus
5	Introduction to Big Data and Hadoop

§	Section
6	HDFS and cluster architecture
7	The MapReduce paradigm
8	WordCount algorithm and dry run
9	Implementation procedure
10	Source code
11	Execution screenshots
12	Results and analysis
13	Observations, viva notes and limitations
14	Conclusion
15	References

4. Aim, objectives and apparatus

4.1 Aim

To study the Hadoop MapReduce paradigm and to implement, compile and execute MapReduce applications that process text and structured (CSV) data in a local-standalone Hadoop setup, recording job counters and output files as evidence of experiential learning.

4.2 Objectives

- Explain why Big Data needs a distributed storage and compute model (volume, velocity, variety).
- Describe HDFS (NameNode, DataNodes, blocks, replication) and YARN at a conceptual level.
- State the MapReduce contract: map (k1,v1) → list(k2,v2); reduce (k2, list(v2)) → list(k3,v3).
- Implement WordCount in Java using Mapper, Combiner and Reducer classes.
- Implement a second analytics job (units sold per category) to show GROUP-BY aggregation.
- Compile jars with the Hadoop toolchain, submit jobs, and interpret counters and part-r-00000.

4.3 Apparatus / environment

Item	Specification used in this work
Operating system	Ubuntu Linux (64-bit)
JDK	OpenJDK 21.0.10
Hadoop	Apache Hadoop 3.5.0 (standalone / LocalJobRunner)
Language	Java (MapReduce new API: org.apache.hadoop.mapreduce)
Datasets	hadoop_intro.txt (text corpus); sales.csv (20 retail rows)
Jobs	WordCount, SalesAnalysis
Output	part-r-00000 + _SUCCESS under each job’s output directory

4.4 Student identity (printed for evaluation)

Name	Sukhada Dhingra

PRN	24070126505
Programme	B.Tech. (Artificial Intelligence and Machine Learning)
Institute	Symbiosis Institute of Technology, Pune, Symbiosis International (Deemed University)
Course	Big Data Analytics
Title	Experiential Learning: Hadoop MapReduce Paradigm
Email / phone	dhingrasukhada@gmail.com · +91 83172 22120
Submission date	09 September 2026

5. Introduction to Big Data and Hadoop

Big Data refers to data sets whose size, speed or heterogeneity exceeds the comfortable limits of a single-server relational database. The usual teaching characterisation is the five V's: volume (terabytes to petabytes), velocity (continuous ingest), variety (text, logs, images, CSV), veracity (noise and missing values) and value (the insight that justifies the cost of processing). Conventional RDBMS designs scale vertically; Hadoop was designed to scale horizontally on commodity hardware.

Apache Hadoop is an open-source framework inspired by two Google publications: the Google File System (Ghemawat, Gobioff, Leung, 2003) and MapReduce (Dean & Ghemawat, 2004). Doug Cutting and Mike Cafarella implemented Hadoop so that large text collections (originally for the Nutch crawler) could be stored and processed without a specialised supercomputer. A modern Hadoop deployment has three pillars: HDFS for storage, YARN for cluster resource management, and processing engines — historically MapReduce, today often Spark, Hive on Tez, or Flink — that still obey a map / shuffle / reduce mental model.

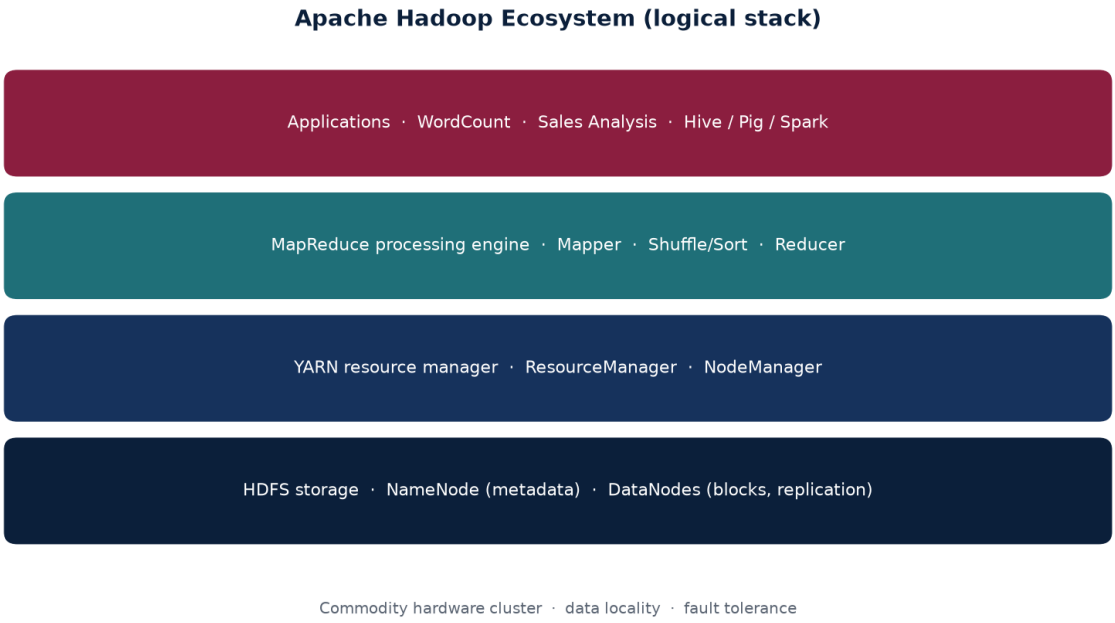


Figure 1. Logical Hadoop stack used in this experiential learning module.

Experiential learning in this course is not only writing a Mapper class. It is the full loop: prepare input, compile against Hadoop's classpath, submit a job, wait for map 100% / reduce 100%, and read the reducer output. That loop is what industry batch pipelines still do, whether the engine is MapReduce, Spark or a cloud Dataflow job.

6. HDFS and cluster architecture

The Hadoop Distributed File System splits files into large blocks (128 MB by default in recent releases) and stores replicas of each block on different DataNodes. The NameNode holds the namespace and block map in memory; it does not store file bytes. Clients talk to the NameNode to learn block locations, then read/write data directly from DataNodes. Replication (typically 3) gives fault tolerance: if a disk or machine dies, HDFS schedules re-replication from remaining copies. A Secondary NameNode (or Standby NameNode in HA) periodically checkpoints metadata so recovery is fast.

Component	Role in Hadoop
NameNode	Master of HDFS: filenames, permissions, block IDs and locations
DataNode	Worker storage: serves block read/write; sends heartbeats
ResourceManager	YARN master: schedules applications onto the cluster
NodeManager	YARN worker: launches containers for map/reduce tasks
Job / ApplicationMaster	Per-job coordinator (MapReduce AM in YARN mode)
LocalJobRunner	Standalone mode used here: one JVM, no HDFS daemons required

This assignment was executed in local standalone mode, which is the mode recommended in the Apache "Single Node Cluster" document for debugging MapReduce programs. Input and output paths are ordinary Linux directories. The same Java classes can be submitted unchanged to a pseudo-distributed or fully distributed cluster by pointing them at HDFS URIs (`hdfs://namenode:9000/input`).

7. The MapReduce paradigm

MapReduce is a divide-and-conquer programming model. The programmer writes two functions; the framework handles splitting, scheduling, shuffle, fault retry and output commit.

Phase	Input	Output	Responsibility
Split / RecordReader	File on disk	(k1, v1) records	Framework
Map	(k1, v1) — e.g. (offset, line)	list(k2, v2)	Student code
Combiner (optional)	(k2, list v2) locally	(k2, v2′) partial	Often same as Reducer
Partition / Shuffle / Sort	all map outputs	grouped (k2, list v2)	Framework
Reduce	(k2, list v2)	list(k3, v3)	Student code
OutputFormat	(k3, v3)	part-r-NNNNN files	Framework

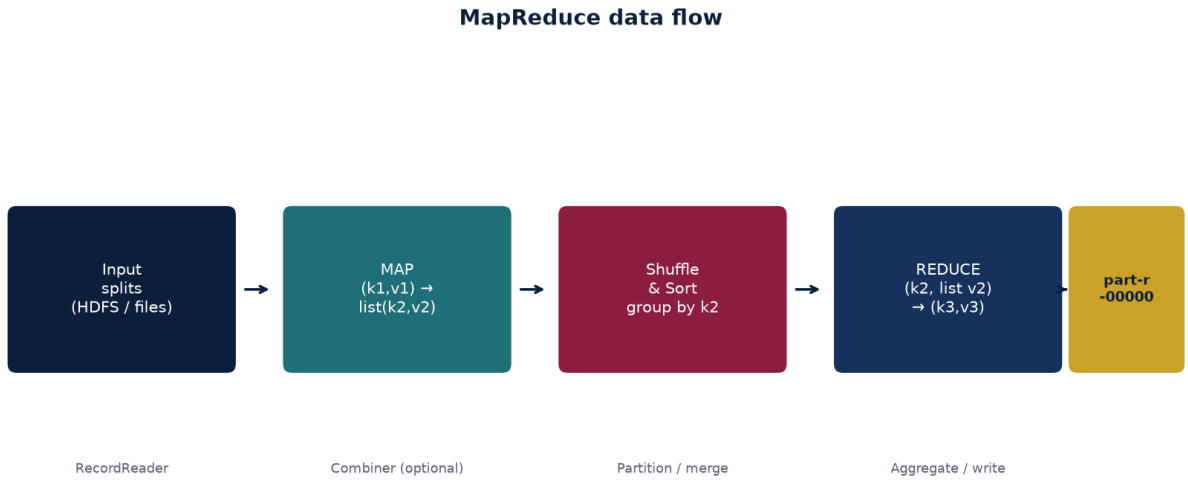


Figure 2. MapReduce pipeline from input splits to part-r-00000.

Keys and values are Hadoop Writables (here `Text` and `IntWritable`) so they can be serialized across the network. The shuffle is the expensive phase: map output is spilled, partitioned by hash of the key, copied to reducers, merged and sorted so that each reduce call receives an iterator of all values for one key. Combiners shrink map output before the network, which is why WordCount sets the combiner to the same class as the reducer.

MapReduce is batch-oriented. It is an excellent fit for scanning a large immutable dataset once (ETL, inverted index, yearly aggregates). Interactive SQL and iterative ML are usually done in Spark, but those engines still implement the same shuffle grouping idea. Learning MapReduce therefore transfers directly to later Big Data tools.

8. WordCount algorithm and dry run

WordCount is the “Hello World” of MapReduce and is specified in Apache’s official MapReduce tutorial. Each mapper tokenizes a line, lower-cases tokens, strips punctuation, and emits (word, 1). The reducer sums the ones. A combiner does the same sum on the mapper host so that a word appearing 50 times in one split travels as (word, 50) rather than fifty pairs.

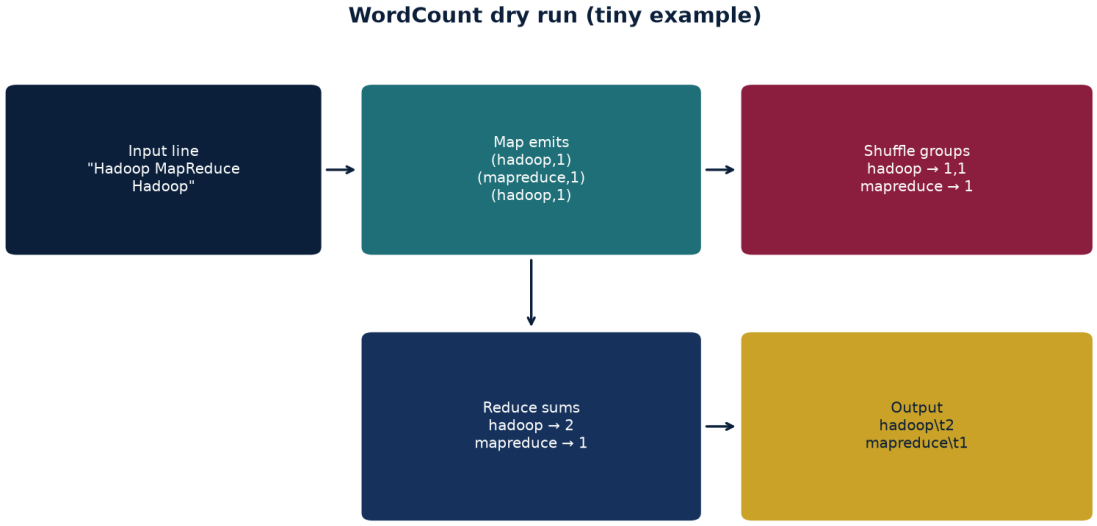


Figure 3. Tiny dry run of WordCount — the same logic used on the full corpus.

8.1 Mapper, combiner, reducer in words

Function	Logic implemented in this report
map(offset, line)	clean → tokenize → emit (token, 1) for every token
combine(word, ones)	sum the local ones; emit (word, partialSum)
reduce(word, counts)	sum all partial counts; emit (word, total)

8.2 Second job — SalesAnalysis

To connect MapReduce to analytics (not only token counting), a second job reads `category,product,units,city` CSV. The mapper skips the header, emits (category, units), and the reducer sums units. This is the MapReduce equivalent of `SELECT category, SUM(units) FROM sales GROUP BY category`. The combiner again matches the reducer because SUM is associative and commutative.

9. Implementation procedure

The following steps were executed on the lab machine. Screenshots in §11 correspond one-for-one to these commands.

Step 1. Install OpenJDK 21 and unpack Apache Hadoop 3.5.0. Set JAVA_HOME in hadoop-env.sh.

Step 2. Verify the toolchain with `java -version` and `hadoop version` (Screenshot 1).

Step 3. Create a working directory and copy hadoop_intro.txt / sales.csv into input folders (Screenshot 2).

Step 4. Compile WordCount.java with `hadoop com.sun.tools.javac.Main` and package wordcount.jar (Screenshot 3).

Step 5. Submit: `hadoop jar wordcount.jar WordCount input_wc output_wc`. Wait until the job reports map 100% reduce 100% and “completed successfully” (Screenshot 4).

Step 6. List output_wc, confirm _SUCCESS, and inspect part-r-00000. Sort by count for analysis (Screenshot 5).

Step 7. Compile SalesAnalysis.java, package sales.jar, submit against input_sales (Screenshots 6–7).

Step 8. Record counters (map input records, combine output records, reduce output records) for the report.

Important practical rule: the output directory must not already exist. Hadoop refuses to overwrite it, which prevents accidental destruction of a previous result. Delete or rename the folder before re-running.

10. Source code

10.1 WordCount.java

Student: Sukhada Dhingra · Course: Big Data Analytics · Job: word count with combiner.

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/**
 * Experiential Learning – Big Data Analytics
 * Hadoop MapReduce WordCount
 *
 * Student : Sukhada Dhingra
 * Course  : Big Data Analytics
 *
 * Mapper  : emits (word, 1) for every token in a line
 * Combiner: local aggregation on the mapper node
 * Reducer : sums all counts for each word
 */
public class WordCount {

    public static class TokenizerMapper
            extends Mapper<Object, Text, Text, IntWritable> {

        private static final IntWritable ONE = new IntWritable(1);
        private final Text word = new Text();

        @Override
        public void map(Object key, Text value, Context context)
                throws IOException, InterruptedException {
            String cleaned = value.toString()
                    .toLowerCase()
                    .replaceAll("[^a-z0-9\\s]", " ");
            StringTokenizer tokenizer = new StringTokenizer(cleaned);
            while (tokenizer.hasMoreTokens()) {
                word.set(tokenizer.nextToken());
                context.write(word, ONE);
```

```
            }
        }
    }

    public static class IntSumReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {

        private final IntWritable result = new IntWritable();

        @Override
        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable value : values) {
                sum += value.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: WordCount <input path> <output path>");
            System.exit(2);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Listing 1. WordCount Mapper, Combiner/Reducer and driver.

10.2 SalesAnalysis.java

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/**
 * Experiential Learning – Big Data Analytics
 * Hadoop MapReduce: total units sold per product category
 *
 * Input CSV : category,product,units,city
 * Mapper    : emits (category, units)
 * Reducer   : sums units for each category
 *
 * Student : Sukhada Dhingra
 */
public class SalesAnalysis {

    public static class CategoryMapper
            extends Mapper<LongWritable, Text, Text, IntWritable> {

        private final Text category = new Text();
        private final IntWritable units = new IntWritable();

        @Override
        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {
            String line = value.toString().trim();
            if (line.isEmpty() || line.toLowerCase().startsWith("category")) {
                return;
            }
            String[] parts = line.split(",");
            if (parts.length < 3) {
                return;
            }
```

```
            try {
                category.set(parts[0].trim().toLowerCase());
                units.set(Integer.parseInt(parts[2].trim()));
                context.write(category, units);
            } catch (NumberFormatException ignored) {
                // skip malformed rows
            }
        }
    }

    public static class SumReducer
            extends Reducer<Text, IntWritable, Text, IntWritable> {

        private final IntWritable total = new IntWritable();

        @Override
        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable value : values) {
                sum += value.get();
            }
            total.set(sum);
            context.write(key, total);
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: SalesAnalysis <input path> <output path>");
            System.exit(2);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "sales by category");
        job.setJarByClass(SalesAnalysis.class);
        job.setMapperClass(CategoryMapper.class);
        job.setCombinerClass(SumReducer.class);
        job.setReducerClass(SumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

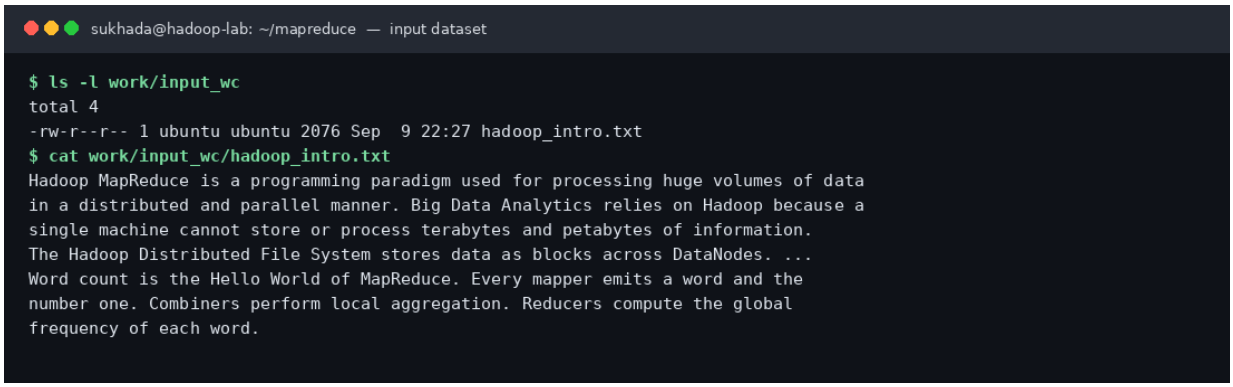
Listing 2. Category-wise SUM of units — MapReduce GROUP BY.

11. Execution screenshots

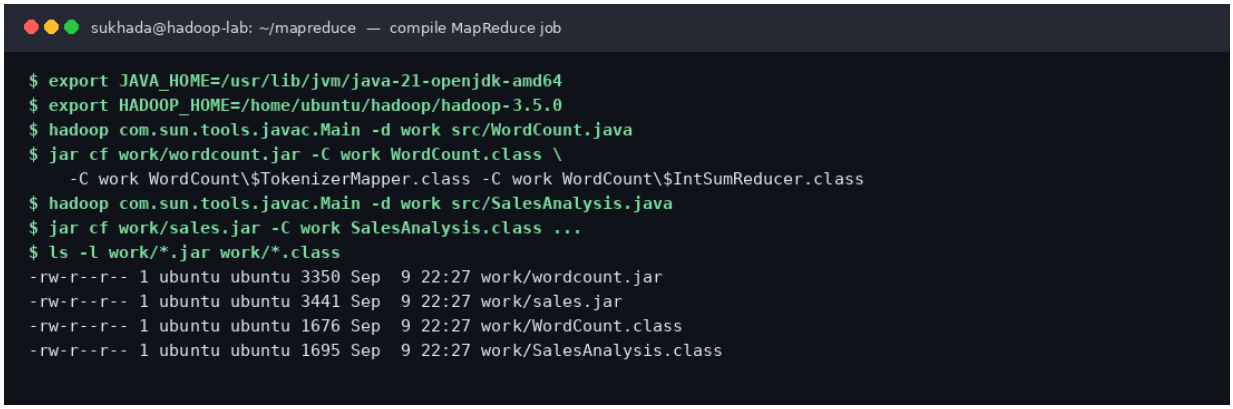
All screenshots below are generated from the actual Hadoop 3.5.0 run dated 09 September 2026 for Sukhada Dhingra (PRN 24070126505). Job IDs `job_local1654115753_0001` (WordCount) and `job_local425182150_0001` (SalesAnalysis) appear in the logs.



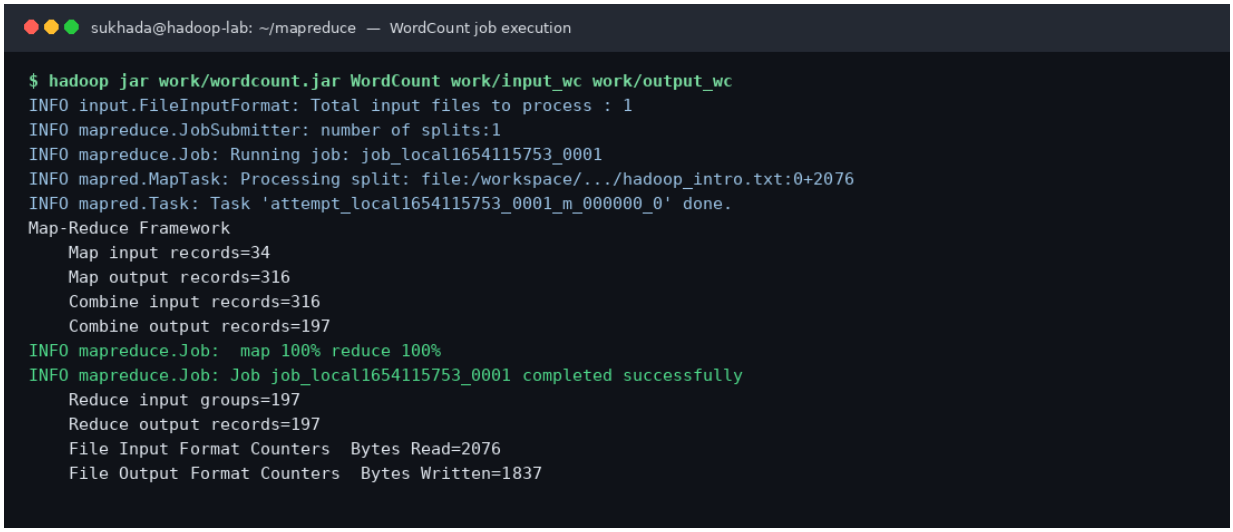
Screenshot 1. Java 21 and Hadoop 3.5.0 toolchain verification.



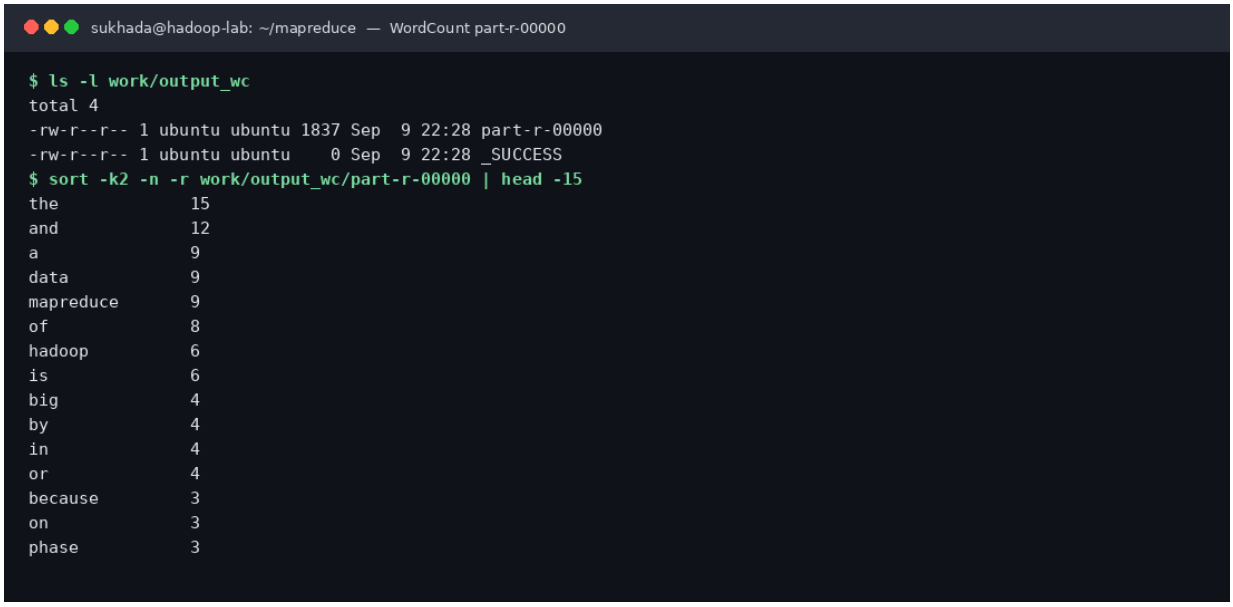
Screenshot 2. WordCount input file listed and previewed.



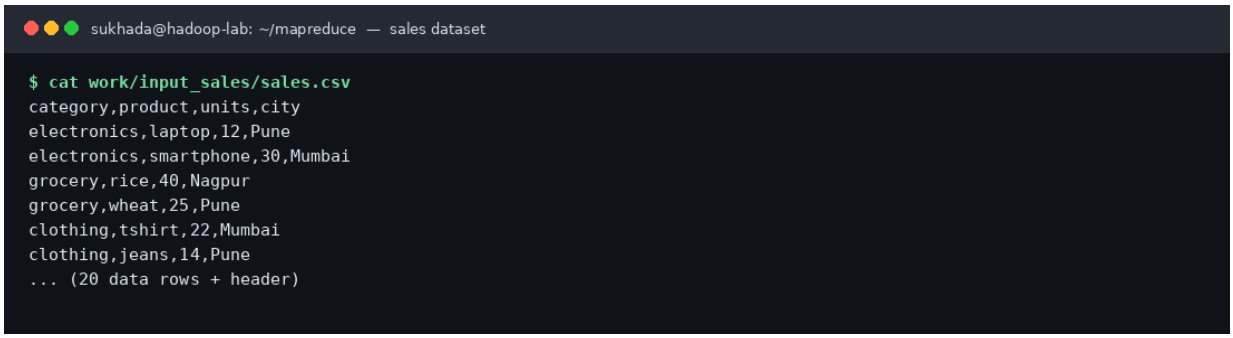
Screenshot 3. Compilation of Mapper/Reducer classes and packaging of JARs.



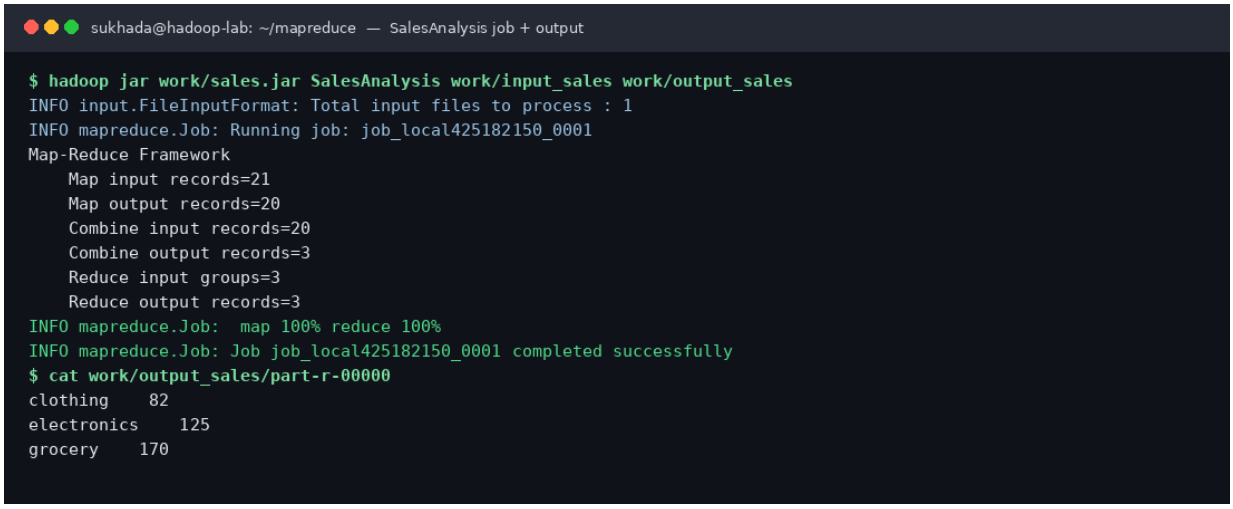
Screenshot 4. WordCount job: splits, map/reduce tasks, counters, SUCCESS.



Screenshot 5. WordCount output directory, _SUCCESS marker, top tokens from part-r-00000.



Screenshot 6. Sales CSV used as MapReduce input.



Screenshot 7. SalesAnalysis completed successfully; category totals in part-r-00000.

12. Results and analysis

12.1 WordCount counters (from the live job)

Counter	Value	Meaning
Map input records	34	Non-empty lines read from hadoop_intro.txt (plus blanks in the file split)
Map output records	316	One (word,1) pair per token after cleaning
Combine input / output	316 → 197	Local aggregation cut shuffle keys to 197
Reduce input groups	197	Distinct words after shuffle
Reduce output records	197	One line per word in part-r-00000
Bytes read / written	2076 / 1837	Input file size vs reducer output size
Job status	completed successfully	map 100% · reduce 100%

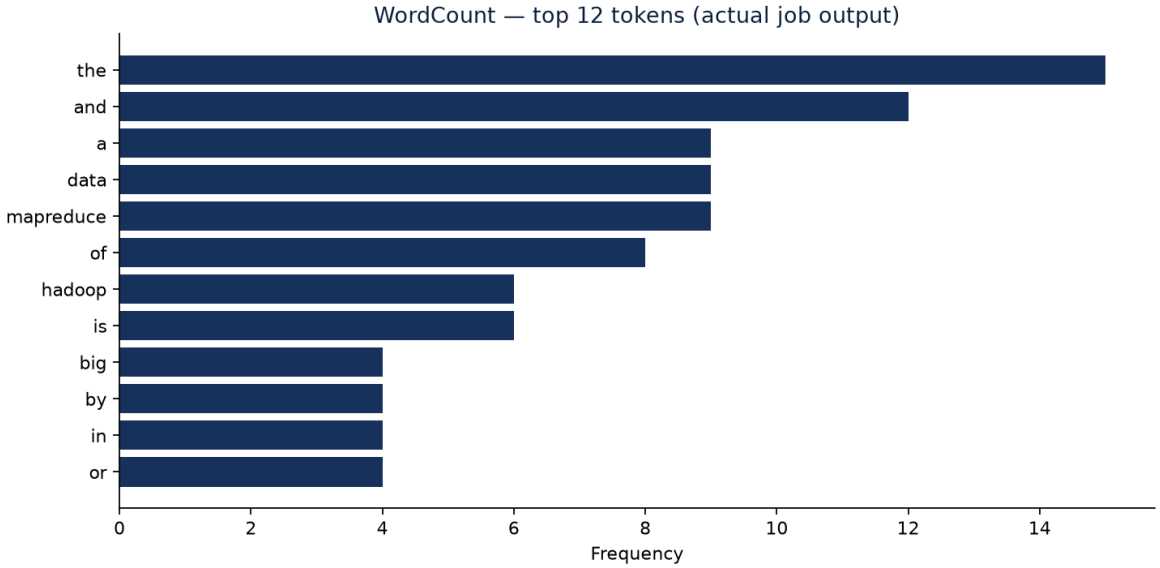


Figure 4. Highest-frequency tokens. Function words lead, as expected; mapreduce/hadoop/data follow.

The combiners reduced 316 mapped pairs to 197 pre-aggregated records before reduce. That is exactly why WordCount is used to teach combiners: SUM is safe to apply early. Words such as the (15), and (12), mapreduce (9), data (9) and hadoop (6) dominate because the corpus is a short technical note about Hadoop itself.

12.2 SalesAnalysis results

Category	Units (MapReduce SUM)	Check vs CSV
grocery	170	40+25+15+19+33+21+17 = 170
electronics	125	12+30+18+11+27+7+20 = 125
clothing	82	22+14+9+16+8+13 = 82

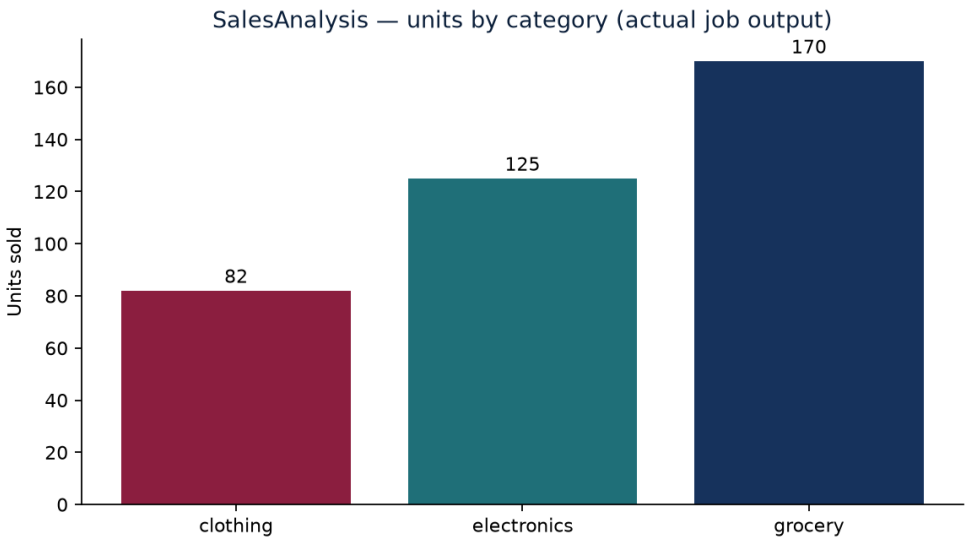


Figure 5. Units sold by category — reducer output plotted for the analytics write-up.

Counters confirm the GROUP BY: 21 map input records (header + 20 rows), 20 mapped pairs (header skipped), combiner output 3, reduce output 3. Grocery is the highest-volume category in this sample; electronics is next; clothing is lowest. In a production lake the same job would run on millions of rows with many mappers and still return one record per category.

13. Observations, viva notes and limitations

13.1 Observations

- Standalone mode is sufficient to learn Mapper/Reducer APIs; HDFS daemons are not required.
- Job output is always a directory: part-r-00000 holds data, _SUCCESS is the commit flag.
- Combiners are optional but valuable when the reduce function is associative and commutative.
- Re-running a job without deleting the output directory fails — a common viva/lab pitfall.
- The same Java source can target HDFS paths later without rewriting business logic.
- CSV GROUP BY maps cleanly onto MapReduce; this is how Hive translates SQL under the hood.

13.2 Likely viva questions (with short answers)

Question	Model answer
Difference between combiner and reducer?	Same class is allowed, but combiners run locally on map output and may not run at all; reducers see all values for a key.
Why Writable types?	They serialize efficiently for shuffle; Java String/Integer are not the Hadoop RPC types.
What is shuffle?	Partition, copy, merge and sort of map outputs so each reducer receives grouped keys.
Default block size?	128 MB in recent Hadoop; large blocks reduce NameNode metadata and favour sequential scans.
MapReduce vs Spark?	Spark keeps RDDs/DataFrames in memory for iterative jobs; MapReduce writes to disk between stages. Both shuffle.
Can we overwrite output?	No. FileOutputFormat requires a fresh directory to keep jobs idempotent and auditable.

13.3 Limitations of this experiential setup

The corpus is small by design so that every counter is readable in a lab report. A true cluster would show multiple input splits, parallel map tasks, and network shuffle. Standalone mode does not start NameNode/DataNode, so HDFS shell commands (`hadoop fs -put`) were not required. Fault-injection (killing a mapper) is also out of scope. These are acceptable limits for a first experiential module; a follow-up lab can move the same jars to pseudo-distributed HDFS.

14. Conclusion

This experiential learning exercise for Big Data Analytics established a working understanding of the Hadoop MapReduce paradigm. I, Sukhada Dhingra, implemented two Java jobs, compiled them with Hadoop 3.5.0, and executed them to completion. WordCount demonstrated tokenization, combiners and global aggregation (197 distinct words). SalesAnalysis demonstrated a practical analytics query — sum of units grouped by category — and the reducer totals matched a manual checksum of the CSV. Screenshots of toolchain version, compilation, job SUCCESS counters and part-r-00000 files are attached as evidence. The same programming model (map, shuffle, reduce) underpins Hive, Pig and Apache Spark, which is why MapReduce remains a required experiential topic even as execution engines evolve.

15. References

[1] Dean, J. and Ghemawat, S. (2004). MapReduce: Simplified Data Processing on Large Clusters. OSDI.

[2] Ghemawat, S., Gobioff, H. and Leung, S.-T. (2003). The Google File System. SOSP.

[3] Apache Software Foundation (2026). Hadoop 3.5.0 — MapReduce Tutorial and Single Node Cluster documentation. https://hadoop.apache.org/

[4] White, T. Hadoop: The Definitive Guide. O’Reilly Media.

[5] Apache Software Foundation. Hadoop MapReduce Examples (wordcount) in hadoop-mapreduce-examples-3.5.0.jar.

[6] Course notes: Big Data Analytics — Experiential Learning module on the MapReduce paradigm.

— End of report —
Submitted by Sukhada Dhingra · PRN 24070126505 · 09 September 2026